

Lesson 8 — YOLO Deep Dive + Data Preparation

Pretrained Model nima?

Pretrained Model — oldindan katta datasetlarda o‘qitib qo‘yilgan tayyor model. “Pretrained” so‘zi “oldindan o‘qitilgan” degani.

Bunday model millionlab rasm ko‘rib, juda ko‘p umumiy visual featurelarni o‘rganib bo‘lgan bo‘ladi. Visual feature — rasm ichidagi foydali belgi yoki xususiyat. Masalan, chiziq, burchak, rang o‘zgarishi, tekstura, ko‘z, burun, g‘ildirak, hayvon yuzi kabi belgilar.

Deep Learning va Computer Visionda modelni har doim noldan o‘qitish shart emas. Chunki katta modelni noldan o‘qitish uchun juda ko‘p data, juda kuchli GPU va juda ko‘p vaqt kerak bo‘ladi. Shu sababli pretrained modeldan boshlash ancha foydali hisoblanadi.

Pretrained model quyidagicha tushuniladi:

Katta datasetlarda oldindan o‘qitilgan model

↓

Umumiy visual featurelarni biladi

↓

Yangi custom datasetga moslashtiriladi

↓

Yangi vazifada ishlaydi

Custom dataset — o‘z loyihasi uchun yig‘ilgan maxsus dataset.

Masalan, faqat baliq turlarini aniqlash, faqat zavoddagi nuqsonli mahsulotlarni topish yoki faqat maxsus kamera rasmlarida obyektlarni aniqlash uchun yig‘ilgan dataset.

GPU — Graphics Processing Unit, ya'ni grafik protsessor. Deep Learning trainingda ko'p matematik hisoblashlarni tez bajarish uchun ishlatiladi.

Training from Scratch

Training from Scratch — modelni noldan o'qitish. “Scratch” bu yerda “noldan boshlash” degani. Bunda model oldindan hech narsa bilmaydi. Uning weightlari tasodifiy qiymatlardan boshlanadi va hamma narsani dataset orqali o'rganishi kerak bo'ladi.

Weight — model ichidagi o'rganiladigan vazn qiymati. Model rasm ichidagi qaysi feature muhimligini weightlar orqali o'rganadi.

Training from Scratch jarayoni quyidagicha:

Random weights

↓

Millionlab rasm

↓

Uzoq training

↓

Model asta-sekin featurelarni o'rganadi

↓

Yakuniy model hosil bo'ladi

Random weights — tasodifiy weightlar. Model boshida chiziq, shakl, obyekt yoki class haqida hech narsa bilmaydi.

Training from Scratchning asosiy kamchiliklari:

millionlab rasm kerak bo'ladi

haftalar yoki oylar davomida GPU vaqti kerak

bo‘ladi

compute xarajati juda katta bo‘ladi

ko‘p tajriba va kuchli texnik bilim talab qilinadi

ko‘p holatda shart emas

Compute xarajati — modelni o‘qitish uchun ketadigan hisoblash resurslari va pul xarajati. Katta modelni kuchli GPU serverlarda o‘qitish qimmat bo‘lishi mumkin.

Training from Scratch asosan juda maxsus domain bo‘lsa ishlatiladi. Domain — ma’lumot turi yoki soha. Masalan, sun’iy yo‘ldosh tasvirlari, tibbiy MRI tasvirlari, sanoat mikroskop rasmlari kabi oddiy rasm datasetlaridan juda farq qiladigan sohalarda noldan training kerak bo‘lishi mumkin.

MRI — Magnetic Resonance Imaging, tibbiy tasvir olish usuli.

Agar millionlab maxsus rasm bo‘lsa va domain oddiy rasmlardan butunlay farq qilsa, modelni noldan o‘qitish mantiqli bo‘lishi mumkin. Lekin ko‘p amaliy loyihalarda bu shart emas.

Pretrained Model va Transfer Learning

Pretrained Modeldan foydalanish ko‘pincha Transfer Learning deb ataladi. Transfer Learning — oldin o‘rganilgan bilimni yangi vazifaga ko‘chirish. “Transfer” — ko‘chirish, “learning” — o‘rganish degani.

Computer Visionda pretrained model avval katta datasetda o‘rganadi. Keyin shu model yangi kichik datasetga moslashtiriladi. Bu usul juda foydali, chunki model oddiy visual featurelarni allaqachon biladi.

Pretrained model bilan ishlash jarayoni:

Katta datasetda o'qitilgan model
↓
Pretrained weights olinadi
↓
Custom dataset tayyorlanadi
↓
Fine-tuning qilinadi
↓
Yangi vazifaga mos model hosil bo'ladi

Pretrained weights — oldindan o'qitilgan modelning saqlangan weightlari. Masalan, YOLOv8 uchun `yolov8n.pt` kabi fayl ishlatilishi mumkin. `.pt` — PyTorch model weight fayli.

Pretrained modelning afzalliklari:

kamroq rasm bilan ishlashi mumkin
training tezroq bo'ladi
Google Colab kabi bepul GPUda ham sinab ko'rish mumkin
yangi boshlovchi uchun qulay
amaliy loyihalarning ko'pida ishlatiladi

Masalan, 200 ta rasm bilan ham pretrained YOLOv8 modelni fine-tuning qilib boshlang'ich yaxshi natija olish mumkin. Bunda model rasmni ko'rish bo'yicha umumiy bilimga ega bo'ladi, custom dataset esa unga faqat yangi vazifani o'rgatadi.

Google Colab — brauzer orqali Python notebook ishlatish va GPUdan foydalanish imkonini beradigan platforma.

Epoch — model butun training datasetni bir marta to‘liq ko‘rib chiqishi. Masalan, 20–50 epoch modelni custom datasetga moslashtirish uchun yetarli bo‘lishi mumkin.

Training from Scratch va Pretrained Model farqi

Training from Scratch va Pretrained Model orasidagi asosiy farq modelning boshlang‘ich bilimida.

Training from Scratchda model hech narsa bilmaydi:

Model boshida random

↓

Chiziqni ham o‘zi o‘rganadi

↓

Shaklni ham o‘zi o‘rganadi

↓

Obyekt qismlarini ham o‘zi o‘rganadi

↓

Katta dataset talab qiladi

Pretrained modelda esa model avvaldan ko‘p narsani biladi:

Model allaqachon chiziq, shakl, tekstura, obyekt qismlarini o‘rgangan

↓

Custom datasetga moslashtiriladi

↓

Kamroq data bilan yaxshi natija beradi

Taqqoslash:

Yoʻnalish	Training from Scratch	Pretrained Model
Boshlanish holati	Random weightlardan boshlanadi	Oldindan oʻqitilgan weightlardan boshlanadi
Data talabi	Juda katta dataset kerak	Kichikroq dataset ham yetishi mumkin
GPU vaqti	Juda koʻp	Ancha kam
Xarajat	Qimmat	Arzonroq yoki bepul Colabda ham mumkin
Tajriba talabi	Yuqori	Boshlangʻich uchun ham qulay
Amaliy ishlatilishi	Maxsus holatlarda	Koʻp loyihalarda

Shu sababli Computer Vision projectlarda odatda pretrained modeldan boshlash toʻgʻri yoʻl hisoblanadi.

Nima uchun Computer Visionda Pretrained Model yaxshi ishlaydi?

Computer Visionda pretrained model yaxshi ishlashining asosiy sababi — rasm ichidagi boshlangʻich visual featurelar koʻp vazifalarda umumiy boʻladi.

Masalan, model ImageNet kabi katta datasetda 1000 ta classni oʻrgangan boʻlsa, u quyidagi narsalarni oʻrganib oladi:

chiziqlar
burchaklar
rang oʻzgarishlari
teksturalar
oddiy shakllar
koʻzga oʻxshash qismlar
burunga oʻxshash qismlar
gʻildirakka oʻxshash qismlar
hayvon tanasi qismlari

ImageNet — juda katta image classification dataset. Unda ko‘plab classlar bor va ko‘plab pretrained modellar shu dataset asosida o‘qitiladi.

Texture — sirt ko‘rinishi yoki naqsh. Masalan, daraxt po‘stlog‘i, mato yuzasi, hayvon juni, metall yuzasi.

Bu featurelar ko‘p rasm turlarida uchraydi. Masalan, chiziq va burchak faqat mushuk rasmda emas, mashina, odam, uy, daraxt, yo‘l belgisi va boshqa ko‘plab obyektlarda ham bor. Shu sababli model oldin o‘rgangan pastki qatlam featurelarini yangi vazifada ham ishlata oladi.

Hayotiy misol sifatida ingliz tilini bilgan kishi ispan tilini o‘rganishini olish mumkin. Til asoslari, grammatik fikrlash, so‘z yodlash usuli va talaffuzga e’tibor berish tajribasi bor bo‘lgani uchun yangi tilni o‘rganish osonlashadi. Computer Visionda ham pretrained model oldin umumiy visual “ko‘rish”ni o‘rgangan bo‘ladi. Keyin yangi custom vazifani tezroq o‘rganadi.

Traditional ML va Deep Learning farqi

Traditional ML — an’anaviy Machine Learning. Masalan, Random Forest, SVM kabi modellar. Bu modellar ko‘pincha feature engineeringga tayanadi.

Feature Engineering — ma’lumotdan qo‘lda yangi foydali featurelar yaratish. Masalan, rasm uchun rang o‘rtachasi, tekstura ko‘rsatkichi, shakl maydoni, kontur uzunligi kabi belgilarni odam hisoblab beradi.

Traditional MLda featurelarni ko‘pincha odam tayyorlaydi:

rang featurelari
shakl featurelari

tekstura featurelari
o'lcham featurelari
statistik featurelar

Random Forest — ko'p decision tree asosida ishlaydigan Machine Learning modeli.

SVM — Support Vector Machine. Classlar orasida eng yaxshi ajratuvchi chegarani topishga harakat qiladigan model.

Traditional MLda pretrained model odatda Computer Visiondagi kabi kuchli ishlaymaydi. Chunki model o'zi rasm ichidan feature o'rganmaydi, featurelarni odam oldindan hisoblab beradi. Har bir dataset uchun featurelar boshqacha bo'lishi mumkin. Masalan, tibbiy rasm uchun kerakli feature bilan meva rasmidagi feature bir xil bo'lmasligi mumkin.

Traditional MLda pretrained yondashuv kamroq foydali bo'lishining sabablari:

featurelarni model emas, odam hisoblaydi
har domain uchun alohida feature kerak bo'ladi
model parameterlari kamroq bo'ladi
transfer qilish samarasi kuchli bo'lmaydi

Deep Learning va CNNda pretrained nima uchun kuchli?

Deep Learningda feature learning avtomatik bo'ladi. Feature learning — modelning o'zi foydali featurelarni topishi. CNN, YOLO, ResNet kabi modellar rasm ichidan chiziq, burchak, shakl, tekstura va obyekt qismlarini avtomatik o'rganadi.

CNN — Convolutional Neural Network. O‘zbekcha izohi: konvolyutsion neyron tarmoq. Rasm bilan ishlashda juda ko‘p ishlatiladi.

Deep Learning modelida qatlamlar quyidagicha o‘rganadi:

Quyi qatlamlar → chiziq, burchak, rang o‘zgarishi
O‘rta qatlamlar → shakl, tekstura, obyekt qismlari
Yuqori qatlamlar → ko‘z, burun, g‘ildirak, yuz, hayvon tanasi

Quyi qatlam featurelari universal bo‘ladi. Universal — ko‘p vazifada umumiy ishlaydigan degani. Chiziq va burchak deyarli hamma rasmda bor. Shu sababli pretrained modelning quyi qatlamlari yangi datasetda ham foydali bo‘ladi.

Deep Learningda pretrained model yaxshi ishlashining sabablari:

featurelar model tomonidan avtomatik o‘rganiladi
quyi qatlam featurelari ko‘p rasmda umumiy
katta model kuchli representation hosil qiladi
faqat oxirgi qatlamlarni moslashtirish ham foyda beradi

Representation — ma’lumotning model ichidagi foydali raqamli ifodasi. Model rasmni oddiy pixel sifatida emas, ma’noli featurelar to‘plami sifatida ifodalaydi.

YOLO va ResNet kabi modellar ko‘p parametrlil bo‘ladi. Parametrlar soni ko‘p bo‘lsa, model murakkab patternlarni o‘rganishi mumkin. Shu sababli pretrained modeldan foydalanish Computer Visionda juda kuchli natija beradi.

Fine-tuning — Pretrained modelni moslashtirish

Fine-tuning — oldindan o‘qitilgan pretrained modelni o‘z datasetga moslashtirish jarayoni. “Fine” — nozik, “tuning” — sozlash degani. Ya’ni modelni butunlay noldan o‘qitmasdan, tayyor modelni yangi vazifaga nozik sozlash.

Fine-tuningda model umumiy ko‘rishni oldindan biladi. Yangi dataset unga o‘z vazifasini o‘rgatadi.

Masalan, YOLO modeli COCO datasetda odam, mashina, it, mushuk kabi ko‘plab obyektlarni o‘rgangan bo‘lishi mumkin. Keyin uni faqat baliq, olcha yoki maxsus mahsulot detectioniga moslashtirish mumkin.

Fine-tuning jarayoni quyidagicha:

ImageNet yoki COCOda pretrained model

↓

Backbone umumiy featurelarni biladi

↓

Custom dataset beriladi

↓

Modelning ayrim yoki barcha qatlamlari qayta o‘rganadi

↓

Custom model hosil bo‘ladi

Fine-tuning oqimi

Fine-tuning chizmasi misol sifatida olinsa, jarayon bir nechta ketma-ket bosqichdan iborat:

ImageNet Pretrained

↓

Backbone Freeze

↓

Faqat Head Train

↓

Full Fine-tune

↓

Your Custom Model

ImageNet Pretrained

ImageNet pretrained — model oldindan katta datasetda o‘qitilgan bo‘ladi. U 1000 class yoki undan ko‘p classlarni ko‘rgan bo‘lishi mumkin. Natijada model umumiy visual featurelarni biladi.

Backbone Freeze

Backbone Freeze — modelning backbone qismi o‘zgarmaydigan qilib qo‘yiladi. Freeze — muzlatish, ya’ni training paytida weightlar o‘zgarmasligi.

Backbone — feature extraction qiladigan asosiy qism. U rasm ichidan chiziq, shakl, tekstura va obyekt qismlarini ajratadi.

Backbone freeze qilinganda modelning eski umumiy visual bilimi saqlanadi. Faqat keyingi prediction qismi o‘rganadi.

Faqat Head Train

Head — modelning prediction chiqaradigan oxirgi qismi. Object Detectionda head bounding box, class va confidence score bashorat qiladi.

Faqat head train qilinsa, modelning feature extraction qismi saqlanadi, lekin yangi classlarni aniqlash uchun prediction qismi moslashadi. Bu juda tez ishlaydi va kichik datasetda foydali bo‘lishi mumkin.

Full Fine-tune

Full Fine-tune — butun modelni kichik learning rate bilan qayta o‘qitish. Learning rate — weight update qadamining kattaligi.

Bu usulda backbone ham, neck ham, head ham ozgina moslashadi. Natija ko‘pincha yaxshiroq bo‘ladi, lekin data ko‘proq kerak bo‘ladi.

Your Custom Model

Your Custom Model — o‘z datasetda ishlaydigan moslashtirilgan model. Masalan, faqat ma’lum obyektlarni topadigan YOLO modeli.

Fine-tuning strategiyalari

Fine-tuningda uchta asosiy strategiya bor:

Feature Extraction

Partial Fine-tuning

Full Fine-tuning

Strategy 1: Feature Extraction

Feature Extraction strategiyasida backbone to‘liq freeze qilinadi. Faqat detection head o‘rganadi.

Feature Extraction — tayyor backbone ajratgan featurelardan foydalanib, faqat oxirgi qismni yangi vazifaga moslash.

Afzalliklari:

eng tez usul

kam data bilan ishlaydi

50–100 rasmda ham boshlang'ich natija olish mumkin

GPU vaqti kam ketadi

Kamchiligi:

model yangi domainga chuqur moslashmasligi mumkin
agar dataset pretrained domaindan juda farq qilsa,
natija cheklangan bo'ladi

Bu usul kichik dataset va tez sinov uchun yaxshi.

Strategy 2: Partial Fine-tuning

Partial Fine-tuning — modelning faqat ayrim qatlamlarini o'qitish.
Masalan, backbone boshidagi qatlamlar freeze qilinadi, oxirgi qatlamlar va head o'rganadi.

Partial — qisman degani.

Bu usul tezlik va aniqlik orasida muvozanat beradi. Model umumiy featurelarni saqlaydi, lekin yangi vazifaga biroz chuqurroq moslashadi.

YOLOda `freeze=10` kabi parametr orqali birinchi qatlamlarni freeze qilish mumkin. Bu modelning boshidagi qatlamlarini o'zgarmas qilib, qolgan qismlarni o'qitishga yordam beradi.

Strategy 3: Full Fine-tuning

Full Fine-tuning — butun modelni o'qitish, lekin odatda kichik learning rate bilan. Bu YOLOv8da default holatga yaqin.

Afzalliklari:

eng yaxshi natija berishi mumkin
model custom datasetga chuqur moslashadi
backbone ham yangi vazifaga mos o'rganadi

Kamchiliklari:

ko'proq data kerak
overfitting xavfi bor
training vaqti ko'proq ketadi

Overfitting — model training data'ni yodlab olib, yangi data'da yomon ishlashi.

Full fine-tuning uchun 200+ rasm kerak bo'lishi mumkin. Dataset qancha sifatli va xilma-xil bo'lsa, fine-tuning shuncha yaxshi ishlaydi.

Image Classification System Architecture: All Steps

Umumiy architecture chizmasi misol sifatida olinsa, Computer Vision project to'rt katta phase orqali productga olib boriladi:

Phase 1: Data Preparation

Phase 2: Model Architecture

Phase 3: Training & Optimization

Phase 4: Deployment & Inference

Bu chizma Image Classification uchun ko'rsatilgan bo'lsa ham, Object Detection va YOLO projectlariga ham juda yaqin mantiq beradi. Farqi shuki, Object Detectionda label faqat class emas, bounding box ham bo'ladi.

Phase 1: Data Preparation

Data Preparation — datasetni modelga tayyorlash bosqichi. Bu qismda rasm yig'iladi, rasm modelga mos formatga keltiriladi va data ko'paytirish ishlari bajariladi.

Phase 1 ichidagi qismlar:

Data Acquisition

Preprocessing

Augmentation

Data Acquisition — rasm yig'ish va label qilish. Acquisition — olish yoki yig'ish degani.

Preprocessing — rasmni modelga mos holatga keltirish. Masalan, resize, normalize, center crop.

Resize — rasm o'lchamini o'zgartirish.

Normalize — pixel qiymatlarini model uchun qulay oraliqqa keltirish.

Center Crop — rasm markazidan kerakli qismni kesib olish.

Augmentation — rasmni turli ko'rinishga keltirib datasetni boyitish. Masalan, flip, rotate, color jitter. Flip — rasmni chap-o'ngga

agʻdarish. Rotate — aylantirish. Color jitter — rang, yorugʻlik va contrastni biroz oʻzgartirish.

Object Detectionda data preparation yanada ehtiyotkorlik bilan qilinadi, chunki rasmga mos label fayl ham kerak boʻladi. Agar rasm resize qilinsa, bounding box koordinatalari ham mos boʻlishi kerak.

Phase 2: Model Architecture

Model Architecture — model ichki tuzilishi. Chizmada model architecture Backbone, Neck va Head qismlariga ajratilgan.

Input Layer

↓

Backbone

↓

Neck

↓

Head

Input Layer — rasm modelga kiradigan qatlam. Masalan, $640 \times 640 \times 3$ oʻlchamdagi RGB rasm.

Backbone — feature extraction qismi. Rasm ichidan muhim featurelarni ajratadi.

Neck — featurelarni birlashtiradi va keyingi prediction qismiga tayyorlaydi.

Head — yakuniy prediction chiqaradi.

Image Classificationda Head class probability chiqaradi. Object Detectionda esa Head bounding box, class probability va confidence score chiqaradi.

Phase 3: Training & Optimization

Training & Optimization modelni o‘qitish va xatosini kamaytirish bosqichi.

Chizmadagi oqim:

Forward Pass
↓
Loss Calculation
↓
Backpropagation
↓
Optimizer
↓
Validation

Forward Pass — input modeldan oldinga qarab o‘tib prediction chiqaradi.

Loss Calculation — model xatosi hisoblanadi. Loss — prediction haqiqiy javobdan qanchalik uzoq ekanini o‘lchaydi.

Backpropagation — xatoni orqaga tarqatib gradientlarni hisoblaydi.

Optimizer — gradientlardan foydalanib weightlarni yangilaydi.

Validation — modelni alohida validation datasetda tekshirish.

Object Detectionda loss faqat class xatosidan iborat bo'lmaydi. Unda box loss, class loss va objectness loss bo'lishi mumkin.

Box loss — bounding box joylashuvi xatosi.

Class loss — class prediction xatosi.

Objectness loss — obyekt bor yoki yo'qligini aniqlash xatosi.

Phase 4: Deployment & Inference

Deployment & Inference — train qilingan modelni real ishlatish bosqichi.

Chizmada bu qism quyidagilardan iborat:

Deployment Optimization

Inference

Monitoring

Final Model

Deployment Optimization — modelni production uchun yengillashtirish. Quantization, ONNX, TensorRT kabi usullar ishlatiladi.

Inference — yangi real data ustida prediction olish.

Monitoring — modelning real tizimda ishlashini kuzatish. Masalan, speed, accuracy, drift va xatolar.

Final Model — foydalanuvchi ishlata oladigan model. Natija probability scores va predicted class ko'rinishida qaytadi.

Object Detectionda final output quyidagicha bo'ladi:

class label
bounding box
confidence score

Masalan:

dog 0.92 [x_center, y_center, width, height]

YOLO Architecture — Backbone

YOLO architecture uchta asosiy qismdan iborat:

Backbone

Neck

Head

Backbone — modelning “ko‘zi” sifatida tushuniladi. U rasm ichidan muhim featurelarni ajratib oladi. YOLO modelidagi parametrlarning katta qismi, taxminan 70–80% atrofidagi qismi backbone ichida bo‘ladi. Pretrained weights asosan shu qismda saqlanadi.

Parameter — model ichidagi o‘rganiladigan qiymat. Weight va bias parametrlari hisoblanadi.

Pretrained weights — oldindan o‘qitilgan modelning saqlangan weightlari.

Input Layer

Backbone input sifatida rasm oladi. YOLOda odatda input rasm quyidagicha bo'lishi mumkin:

$640 \times 640 \times 3$

Bu yerda:

640 → rasm balandligi

640 → rasm kengligi

3 → RGB kanal

RGB — Red, Green, Blue. Qizil, yashil va ko'k rang kanallari. Rangli rasm 3 ta kanal orqali ifodalanadi.

Kompyuter rasmni oddiy ko'z bilan ko'rgandek ko'rmaydi. U rasmni pixel qiymatlari sifatida oladi. Pixel — rasmning eng kichik nuqtasi.

Quyi qatlamlar

Backbonening quyi qatlamlari oddiy featurelarni o'rganadi:

chiziqlar

burchaklar

rang o'zgarishlari

oddiy qirralar

Bular har qanday rasmda ko'p uchraydi. Shu sababli pretrained modelning pastki qatlamlari boshqa vazifalarga ham ko'chirilishi mumkin.

Masalan, mushuk, mashina, odam, yo‘l belgisi yoki tibbiy rasmda ham chiziq va burchaklar mavjud bo‘ladi.

O‘rta qatlamlar

O‘rta qatlamlar murakkabroq featurelarni o‘rganadi:

shakllar

teksturalar

obyekt qismlari

takrorlanuvchi patternlar

Pattern — takrorlanadigan belgi yoki tuzilma.

Texture — sirt ko‘rinishi. Masalan, hayvon juni, mato, metall, daraxt po‘stlog‘i.

Bu qatlamlarda model rasm ichidagi oddiy chiziqlarni birlashtirib, kattaroq ma’noli qismlarni topa boshlaydi.

Yuqori qatlamlar

Yuqori qatlamlar obyektga yaqin featurelarni o‘rganadi:

ko‘z

quloq

g‘ildirak

burun

hayvon yuzi

avtomobil qismi

odam tanasi qismi

Bu featurelar detection uchun juda muhim. Chunki model obyektning faqat rang orqali emas, uning qismlari va shakli orqali ham taniydi.

Masalan, mashinani topishda g'ildirak, faralar, oyna, kuzov shakli muhim bo'lishi mumkin. Itni topishda quloq, tumshuq, tana shakli muhim bo'ladi.

Feature Maps

Backbone oxirida feature maps hosil bo'ladi. Feature map — model rasm ichidan ajratgan featurelarning xaritasi.

YOLOda multi-scale feature maps ishlatiladi:

52×52

26×26

13×13

Multi-scale — bir nechta o'lchamda. Bu kichik, o'rta va katta obyektlarni aniqlashga yordam beradi.

Katta feature map kichik obyektlarni ko'rishga yordam beradi. Chunki unda spatial detail, ya'ni joylashuv tafsilotlari ko'proq saqlanadi.

Kichik feature map katta obyektlar va umumiy kontekstni yaxshiroq ushlaydi.

Spatial detail — rasm ichidagi joylashuv tafsiloti.

YOLOv8 Backbone: CSPDarknet53 va C2f modules

YOLOv8 backbone qismida CSPDarknet va C2f kabi module g'oyalari ishlatiladi.

CSPDarknet53 — YOLOv5da ishlatilgan backbone turi. CSP — Cross Stage Partial. Bu feature reuse va kamroq parametr bilan samarali feature extraction qilishga yordam beradi.

Feature reuse — oldingi qatlamlarda olingan featurelardan keyingi qatlamlarda qayta foydalanish.

C2f — YOLOv8da ishlatiladigan CSP Bottleneck turidagi module. U gradient oqimini yaxshilashga yordam beradi.

Gradient — lossni kamaytirish uchun weight qaysi tomonga o'zgarishi kerakligini ko'rsatadigan qiymat.

Gradient flow — gradientlarning network bo'ylab yaxshi o'tishi. Gradient flow yaxshi bo'lsa, model chuqur qatlamlarda ham yaxshiroq o'rganadi.

EfficientNet — EfficientDetda ishlatiladigan backbone. Compound scaling orqali model kengligi, chuqurligi va rasm resolutionini birgalikda moslaydi.

Compound scaling — model width, depth va resolutionini muvozanatli kattalashtirish.

Width — model qatlamlaridagi channel soni.

Depth — qatlamlar chuqurligi.

Resolution — rasm o'lchami.

ResNet50/101 — Faster R-CNN kabi modellar bilan ishlatilishi mumkin. ResNet skip connections orqali vanishing gradient muammosini kamaytiradi.

Skip connection — oldingi qatlamdan keyingi qatlamga to‘g‘ridan-to‘g‘ri ulanish.

Vanishing gradient — gradient juda kichiklashib, chuqur qatlamlar yaxshi o‘rganmay qolishi muammosi.

Ultralytics va YOLOv8 Platformasi

Ultralytics — YOLOv5 va YOLOv8 bilan bog‘liq mashhur Computer Vision framework. Framework — modelni train qilish, predict qilish, export qilish va deploy qilish uchun tayyor vositalar to‘plami.

Ultralytics platformasi Object Detection bilan birga boshqa vazifalarni ham qo‘llab-quvvatlaydi:

Object Detection
Instance Segmentation
Pose Estimation
Image Classification
Object Tracking

Object Detection — obyektни va uning joylashuvini topish.

Instance Segmentation — har bir obyektни pixel mask bilan alohida ajratish.

Pose Estimation — odam tanasidagi keypointlarni topish. Keypoint — tana nuqtasi, masalan yelka, tirsak, tizza.

Image Classification — rasmni classga ajratish.

Object Tracking — videoda obyektни frame'dan frame'ga kuzatish.

Ultralytics bepul va open-source sifatida ishlatiladi. Open-source — kod ochiq, o'rganish va foydalanish mumkin degani.

Nima uchun YOLOv8 qulay?

YOLOv8ning asosiy qulayliklaridan biri kam kod bilan ishlashi.

Umumiy flow quyidagicha:

```
pip install  
↓  
YOLO()  
↓  
.train()  
↓  
.predict()
```

`pip install` — Python kutubxonasini o'rnatish buyrug'i.

`YOLO()` — YOLO model obyektini yaratish.

`.train()` — modelni training qilish.

`.predict()` — modeldan prediction olish.

Bu beginnerlar uchun juda qulay, chunki model training va inference jarayonini qo'lda murakkab qilib yozish shart emas.

Anchor-free arxitektura

YOLOv8 anchor-free architecture ishlatadi. Anchor-free — oldindan belgilangan anchor boxlarsiz ishlash.

Anchor box — oldindan belgilangan box o'lchami. Eski detectorlarda model shu anchor boxlar atrofida bounding boxni taxmin qilgan.

Anchor-free yondashuvda model obyekt markazi va o'lchamini to'g'ridan-to'g'ri bashorat qiladi. Bu yondashuv soddaroq, tezroq va sozlash osonroq bo'lishi mumkin.

Bashorat qilish — prediction qilish.

Anchor-free afzalliklari:

anchor box tanlash shart emas
model sozlash osonlashadi
training jarayoni soddalashadi
turli o'lchamdagi obyektlar bilan ishlash
qulaylashadi

YOLOv8 model o'lchamlari

YOLOv8 bir nechta model o'lchamlariga ega:

n
s
m
l
x

Bu harflar modelning kattaligini bildiradi:

n → nano
s → small
m → medium
l → large
x → xlarge

Kichik model tezroq ishlaydi, lekin aniqlik pastroq bo‘lishi mumkin.
Katta model aniqroq bo‘ladi, lekin sekinroq va og‘irroq ishlaydi.

Export formatlar

YOLOv8 ko‘p formatga export qilinishi mumkin:

ONNX
TFLite
CoreML
TensorRT

Export — modelni boshqa formatga chiqarish.

ONNX — Open Neural Network Exchange. Turli framework va runtime orasida modelni ko‘chirish uchun universal format.

TFLite — TensorFlow Lite. Mobil va edge device uchun yengil inference formati.

CoreML — Apple qurilmalarida Machine Learning model ishlatish formati.

TensorRT — NVIDIA GPUda inference tezligini oshiradigan engine.

Bu formatlar modelni notebookdan tashqarida, real application yoki device ichida ishlatishga yordam beradi.

YOLO Architecture — Neck va Head

YOLO architecture ichida Backbone featurelarni ajratadi. Lekin faqat backbone yetarli emas. Ajratilgan featurelar birlashtirilishi va predictionga aylanishi kerak. Shu vazifani Neck va Head bajaradi.

Neck — Feature Aggregator

Neck — feature aggregator. Aggregator — birlashtiruvchi degani. Neck backbone'dan kelgan turli scale'dagi feature maplarni oladi va ularni birlashtiradi.

Scale — o'lcham darajasi. Masalan, katta feature map, o'rta feature map, kichik feature map.

Backbone'dan 3 xil scale'dagi feature map kelishi mumkin:

52 × 52

26 × 26

13 × 13

Neck shu feature maplarni birlashtirib, modelga kichik va katta obyektlarni birgalikda ko'rishga yordam beradi.

Kichik obyektlar uchun yuqori resolution feature maplar muhim. Katta obyektlar uchun esa chuqur va kontekstli featurelar muhim.

Resolution — rasm yoki feature mapning o'lcham aniqligi.

Context — obyekt atrofidagi umumiy vaziyat yoki ma'no.

FPN — Feature Pyramid Network

FPN — Feature Pyramid Network. O‘zbekcha izohi: feature piramidasi tarmog‘i.

FPN yuqoridan pastga ma’lumot uzatadi:

katta semantik featurelar

↓

kichik obyektlarni ko‘rishga yordam beradigan featurelar

Semantic feature — chuqur ma’noli feature. Masalan, “bu hayvonga o‘xshaydi”, “bu transportga o‘xshaydi” kabi yuqoriroq darajadagi belgi.

FPN kichik obyektlarni aniqlashda foydali. Chunki kichik obyektlar katta rasmda kam joy egallaydi va ularni topish qiyinroq bo‘ladi.

PAN — Path Aggregation Network

PAN — Path Aggregation Network. U pastdan yuqoriga ham ma’lumot uzatadi. Bu katta obyektlarni ham yaxshi ko‘rishga yordam beradi.

PAN FPNdan kelgan ma’lumot oqimini boyitadi. Pastki qatlamlardagi joylashuv tafsilotlari yuqori qatlamlardagi ma’no bilan birlashadi.

YOLOv8da FPN + PAN ishlatiladi. Bu PANet deb tushuniladi.

PANet — featurelarni yuqoridan pastga va pastdan yuqoriga birlashtirishga yordam beradigan network.

Head — Prediction Layer

Head — prediction layer. Prediction Layer — model yakuniy bashorat chiqaradigan qatlam.

YOLO Head har bir grid cell uchun prediction qiladi. Grid cell — rasmni kichik katakchalarga bo'lingandagi bitta katak.

Head quyidagilarni bashorat qiladi:

bounding box
objectness score
class probabilities

Bounding box — obyekt joylashuvini ko'rsatadi.

Objectness score — shu joyda obyekt borligiga model ishonchi.

Class probabilities — obyekt qaysi classga tegishli ekaniga ehtimollar.

YOLOv8 Head output formati:

$S \times S \times (4 + 1 + C)$

Bu yerda:

$S \times S \rightarrow$ grid o'lchami
 $4 \rightarrow$ bbox coordinates, ya'ni x, y, w, h
 $1 \rightarrow$ objectness score yoki confidence
 $C \rightarrow$ class probabilities soni

Agar COCO datasetida 80 class bo'lsa, $C = 80$ bo'ladi.

Misol:

$$\begin{aligned}
 &13 \times 13 \times (4 + 1 + 80) \\
 &= 13 \times 13 \times 85 \\
 &= 14,365 \text{ bashorat}
 \end{aligned}$$

Bu yerda 13×13 gridga har bir joy uchun 85 ta qiymat chiqadi. 85 qiymat ichida 4 ta bbox koordinata, 1 ta objectness score va 80 ta class probability bor.

Bu Head qismi Object Detection natijasini hosil qiladi. Backbone va Neck featurelarni tayyorlaydi, Head esa ularni bounding box va class predictionga aylantiradi.

YOLOv8 Model Sizes — n / s / m / l / x

YOLOv8da modelning bir nechta o'lchamlari bor. Model tanlashda tezlik, aniqlik, fayl hajmi va qurilma imkoniyati hisobga olinadi.

Asosiy tushunchalar:

Params

mAP

FPS

File size

Deployment environment

Params — model parameterlari soni. Parameter qancha ko'p bo'lsa, model odatda kuchliroq bo'ladi, lekin sekinroq va og'irroq ishlaydi.

mAP — mean Average Precision. Object Detectionda asosiy aniqlik metriclaridan biri.

FPS — Frames Per Second. Model sekundiga nechta rasm yoki frame qayta ishlay olishini bildiradi.

File size — model faylining hajmi.

Deployment environment — model ishlaydigan muhit. Masalan, telefon, Raspberry Pi, server, Colab GPU, A100 GPU.

YOLOv8n — nano

YOLOv8n — nano model. Eng yengil variantlardan biri.

Koʻrsatkichlari:

Params: 3.2M

mAP: 37.3

FPS: 200+

File: 6.3MB

M — million degani. 3.2M — 3.2 million parametr.

YOLOv8n telefon, Raspberry Pi, IoT va Edge device uchun mos.

IoT — Internet of Things, internetga ulangan kichik qurilmalar.

Edge device — hisoblash qurilmaning oʻzida bajariladigan muhit.

Masalan, smart camera, Raspberry Pi, telefon.

YOLOv8n juda tez ishlaydi, lekin aniqligi katta modellarga qaraganda pastroq boʻlishi mumkin. Boshlangʻich oʻrganish va tez test qilish uchun juda qulay.

YOLOv8s — small

YOLOv8s — small model. Nano modeldan kattaroq, lekin hali ham yengil.

Ko'rsatkichlari:

Params: 11.2M

mAP: 44.9

FPS: 120

File: 21.5MB

YOLOv8s yengil server, embedded camera va kichik production tizimlar uchun mos.

Embedded camera — ichiga AI yoki processing joylashtirilgan kamera.

YOLOv8s YOLOv8nga qaraganda aniqroq, lekin biroz sekinroq. Ko'p o'rganish projectlari uchun yaxshi balans beradi.

YOLOv8m — medium

YOLOv8m — medium model. Tezlik va aniqlik orasida muvozanatli variant.

Ko'rsatkichlari:

Params: 25.9M

mAP: 50.2

FPS: 80

File: 49.7MB

YOLOv8m Colab GPUda yaxshi ishlashi mumkin. Bu model juda kichik emas, juda katta ham emas. Aniqlik kerak, lekin model juda og'ir bo'lmasin deyilsa, YOLOv8m yaxshi tanlov bo'ladi.

YOLOv8l — large

YOLOv8l — large model. Katta va aniqroq model.

Ko'rsatkichlari:

Params: 43.7M

mAP: 52.9

FPS: 50

File: 83.7MB

YOLOv8l yuqori aniqlik kerak bo'lgan vazifalarda ishlatiladi. Kuchli GPU kerak bo'ladi. Fayl hajmi ham kattaroq.

YOLOv8x — xlarge

YOLOv8x — eng katta variantlardan biri.

Ko'rsatkichlari:

Params: 68.2M

mAP: 53.9

FPS: 30

File: 130.5MB

YOLOv8x eng yuqori aniqlik uchun ishlatiladi. Lekin tezlik pastroq va model fayli katta. A100 GPU kabi kuchli GPU muhitlari uchun mos.

A100 GPU — NVIDIAning kuchli data center GPUlaridan biri.

Model tanlash bo'yicha xulosa

YOLOv8 model size tanlashda quyidagi qoida ishlatiladi:

tez o'rganish va test → YOLOv8n

yaxshi natija va yengil model → YOLOv8s

muvozanatli variant → YOLOv8m

yuqori aniqlik → YOLOv8l

eng yuqori aniqlik va kuchli GPU → YOLOv8x

Kurs va beginner projectlar uchun YOLOv8n bilan boshlash qulay.

Keyin natija yaxshiroq kerak bo'lsa YOLOv8s ishlatish mumkin.

Colab GPUda YOLOv8m ham yaxshi variant bo'lishi mumkin.

YOLO Dataset Formati — Papka va Label fayllari

YOLO modelni custom datasetda train qilish uchun dataset to'g'ri formatda bo'lishi kerak. Object Detection datasetida har bir rasm uchun label fayl bo'ladi. Label faylda class id va bounding box koordinatalari yoziladi.

YOLO dataset structure odatda quyidagicha bo'ladi:

```
dataset/
├── images/
│   ├── train/
│   │   ├── dog001.jpg
│   │   └── dog002.jpg
│   ├── val/
│   │   └── dog100.jpg
│   └── test/ (optional)
├── labels/
│   ├── train/
│   │   ├── dog001.txt
│   │   └── dog002.txt
│   └── val/
│       └── dog100.txt
└── data.yaml
```

images folder

images folder ichida rasmlar saqlanadi. U train, val va test qismlariga ajratiladi.

Train — model o‘rganadigan rasm to‘plami.

Val — validation, ya’ni training paytida modelni tekshirish uchun ishlatiladigan rasm to‘plami.

Test — yakuniy baholash uchun ishlatiladigan rasm to‘plami.

Optional — majburiy emas, ixtiyoriy degani.

labels folder

`labels` folder ichida har bir rasmga mos `.txt` label fayl turadi.

Rasm nomi va label fayl nomi bir xil bo'lishi kerak.

Masalan:

`dog001.jpg` → `dog001.txt`

`dog002.jpg` → `dog002.txt`

`dog100.jpg` → `dog100.txt`

Agar rasm `images/train` ichida bo'lsa, label ham `labels/train` ichida bo'lishi kerak.

Agar rasm `images/val` ichida bo'lsa, label ham `labels/val` ichida bo'lishi kerak.

Bu moslik juda muhim. Agar rasm bor, lekin label yo'q bo'lsa, trainingda xato chiqishi yoki model noto'g'ri o'rganishi mumkin.

YOLO Label fayl formati

YOLO label fayli `.txt` ko'rinishida bo'ladi. Har bir qator bitta obyektни bildiradi.

Format:

`class_id x_center y_center width height`

Misol:

`0 0.50 0.45 0.30 0.60`

Bu bitta obyekt borligini bildiradi.

class_id

class_id — classning raqamli idsi. YOLO label faylda class nomi yozilmaydi, class raqami yoziladi.

Masalan:

```
dog = 0  
cat = 1  
bird = 2
```

Bu tartib `data.yaml` faylidagi `names` ro'yxatiga bog'liq bo'ladi.

Agar `names: ['dog', 'cat', 'bird']` bo'lsa:

```
0 → dog  
1 → cat  
2 → bird
```

x_center

x_center — bounding box markazining x koordinatasi. YOLO formatda bu normalized qiymatda bo'ladi. Normalized — 0 dan 1 gacha oraliqqa keltirilgan.

Misol:

0.50

Bu box markazi rasm kengligining 50% joyida, ya'ni taxminan markazda ekanini bildiradi.

y_center

y_center — box markazining y koordinatasi. Bu ham normalized bo'ladi.

Misol:

0.45

Bu box markazi rasm balandligining yuqoridan 45% qismida ekanini bildiradi.

width

width — box kengligi.

Misol:

0.30

Bu box rasm kengligining 30% qismini egallaydi degani.

height

height — box balandligi.

Misol:

0.60

Bu box rasm balandligining 60% qismini egallaydi degani.

YOLO labeldagi normalized qiymatlar

YOLO label formatida koordinatalar 0–1 oraliqda bo‘ladi. Bu juda muhim.

Agar rasm 1000 pixel kenglikda bo‘lsa va box markazi 500-pixelda bo‘lsa:

$$x_center = 500 / 1000 = 0.50$$

Agar box kengligi 300 pixel bo‘lsa:

$$width = 300 / 1000 = 0.30$$

Bunday normalized format turli o‘lchamdagi rasmlar bilan ishlashni qulay qiladi.

Bitta rasmda bir nechta obyekt bo‘lsa

Agar bitta rasmda bir nechta obyekt bo‘lsa, label faylda bir nechta qator bo‘ladi.

Masalan:

```
0 0.50 0.45 0.30 0.60
1 0.20 0.30 0.10 0.25
2 0.70 0.55 0.15 0.20
```

Bu 3 ta obyekt borligini bildiradi:

0-class obyekt

1-class obyekt

2-class obyekt

Agar names: ['dog', 'cat', 'bird'] bo'lsa:

dog
cat
bird

data.yaml — Dataset config

data.yaml — YOLO dataset konfiguratsiya fayli. Config — sozlama fayli. Bu fayl YOLOga dataset qayerda ekanini, nechta class borligini va class nomlarini aytadi.

Oddiy ko'rinish:

```
path: ./dataset
train: images/train
val: images/val
nc: 3
names: ['dog', 'cat', 'bird']
```

path — dataset asosiy papkasi.

train — train rasmlar papkasi.

val — validation rasmlar papkasi.

nc — number of classes, ya'ni classlar soni.

names — class nomlari ro'yxati.

Agar `nc` va `names` mos kelmasa, xato bo'lishi mumkin. Masalan, `nc: 3` bo'lsa, `names` ichida 3 ta class bo'lishi kerak.

To'g'ri:

```
nc: 3
names: ['dog', 'cat', 'bird']
```

Noto'g'ri:

```
nc: 3
names: ['dog', 'cat']
```

Bu xato, chunki classlar soni 3 deyilgan, lekin nomlar 2 ta.

Annotation vositalari: LabelImg, Roboflow, CVAT

Annotation — rasm ichidagi obyektни belgilash jarayoni. Object Detectionda annotation obyekt atrofida bounding box chizish va unga class nomi berishdan iborat.

Masalan, rasmda it bo'lsa:

```
it atrofida box chiziladi
class nomi dog deb belgilanadi
YOLO formatda txt label fayl saqlanadi
```

Annotation tool — rasmga box va label qo'yishga yordam beradigan dastur.

Asosiy vositalar:

LabelImg

Roboflow

CVAT

LabelImg

LabelImg — desktop, ya'ni kompyuterga o'rnatib ishlatiladigan annotation tool.

Desktop — lokal kompyuterda ishlaydigan dastur.

Offline — internet bo'lmasa ham ishlashi mumkin.

LabelImg afzalliklari:

bepul

open-source

YOLO formatni to'g'ridan-to'g'ri qo'llaydi

o'rnatish oson

beginner uchun tushunarli

Open-source — kodi ochiq va bepul foydalanish mumkin.

O'rnatish:

```
pip install labelImg
```

Ishga tushirish:

```
labelImg
```

LabelImg kamchiliklari:

har bir rasm qo'lda annotatsiya qilinadi
batch yoki auto-label imkoniyati yo'q
interfeysi eski
katta datasetda sekin bo'lishi mumkin

Batch — bir nechta faylni birga qayta ishlash.

Auto-label — AI yordamida avtomatik label qo'yish.

LabelImg kichik dataset, kurs ishlari va boshlang'ich amaliyot uchun juda mos.

Roboflow

Roboflow — online cloud annotation va dataset management platformasi.

Online cloud — internet orqali ishlaydigan platforma.

Roboflow afzalliklari:

auto-labeling bor
YOLO, COCO, VOC kabi ko'p formatga export qiladi
augmentation avtomatik
datasetni boshqarish qulay
web orqali ishlaydi

Auto-labeling — AI yordamida obyektlarga avtomatik label qo'yish.

Export — datasetni kerakli formatda chiqarish.

COCO va VOC — mashhur annotation formatlari.

Roboflow workflow:

New Project

↓

Upload

↓

Annotate

↓

Export

New Project — yangi project yaratish.

Upload — rasm yuklash.

Annotate — rasmga box va label qo'yish.

Export — datasetni YOLO yoki boshqa formatda yuklab olish.

Roboflow kamchiliklari:

internet kerak

katta dataset uchun pullik bo'lishi mumkin

maxfiy data uchun ehtiyot bo'lish kerak

Roboflow katta loyiha, auto-label, augmentation va export kerak bo'lganda juda qulay.

CVAT

CVAT — professional darajadagi annotation tool. CVAT “Computer Vision Annotation Tool” degani.

CVAT web yoki self-hosted ko'rinishda ishlatilishi mumkin.

Self-hosted — o‘z serveriga o‘rnatib ishlatish.

CVAT afzalliklari:

professional annotation uchun kuchli
video annotation qo‘llab-quvvatlaydi
model-assisted labeling bor
jamoalar bilan ishlashga mos
katta projectlar uchun qulay

Video annotation — video ichidagi frame’larda obyektlarni belgilash.

Model-assisted labeling — model yordamida label qo‘yishni tezlashtirish.

CVAT kamchiliklari:

o‘rnatish murakkabroq
yangi boshlovchilar uchun qiyinroq
interfeys va sozlamalarni tushunish vaqt oladi

Beginner uchun LabelImg eng sodda. Katta project uchun Roboflow qulay. Professional yoki video annotation kerak bo‘lsa CVAT foydali.

Manual va Automatic Annotation

Real loyihalarda annotation faqat qo‘lda yoki faqat avtomatik qilinmaydi. Ko‘p hollarda ikkalasi birga ishlatiladi. Bu yondashuv Human-in-the-Loop deb ataladi.

Human-in-the-Loop — jarayonda odam ham, model ham ishtirok etadigan yondashuv. Model yordam beradi, lekin odam tekshiradi va xatolarni tuzatadi.

Human-in-the-Loop jarayoni

Human-in-the-Loop jarayoni bosqichlari:

1. 200 rasm manual annotation qilinadi
2. Kichik model train qilinadi
3. Model qolgan rasmlarni auto annotation qiladi
4. Odam xatolarni tekshiradi va tuzatadi
5. Full dataset bilan retrain qilinadi

1. Manual annotation

Avval kichik dataset qo'lda annotatsiya qilinadi. Masalan, 200 ta rasm LabelImg yoki Roboflow bilan belgilab chiqiladi.

Manual — qo'lda bajarilgan.

Bu bosqichda sifat juda muhim. Chunki birinchi model shu labeldan o'rganadi.

2. Kichik model train

Keyin YOLOv8n kabi yengil model 30 epoch atrofida train qilinadi.

YOLOv8n — YOLOv8ning nano varianti, tez ishlaydi.

Bu model hali mukammal bo'lmaydi, lekin qolgan rasmlarni taxminiy annotation qilishga yordam beradi.

3. Auto annotation

Train qilingan kichik model qolgan rasmlardagi obyektlarni avtomatik belgilaydi.

Auto annotation — model yordamida avtomatik box va class berish.

Bu qoʻlda annotation vaqtini ancha kamaytiradi.

4. Odam tekshiradi

Model xato qilishi mumkin. Shu sababli odam auto annotation natijasini tekshiradi.

Tuzatiladigan holatlar:

- box notoʻgʻri joylashgan
- class notoʻgʻri berilgan
- obyekt topilmagan
- ortiqcha box chizilgan
- kichik obyekt oʻtkazib yuborilgan

Odam faqat xatolarni tuzatgani uchun jarayon toʻliq manual annotationga qaraganda tezroq boʻladi.

5. Retrain

Tozalangan full dataset bilan model qayta train qilinadi.

Retrain — modelni qayta oʻqitish.

Bu bosqichdan keyin model ancha yaxshilanadi. Keyin yana auto annotation, tekshirish va retrain jarayoni takrorlanishi mumkin.

Real kompaniyalarda annotation yondashuvlari

Real loyihalarda annotation turli ko‘rinishda bo‘ladi.

Tesla kabi self-driving kompaniyalarda millionlab real yo‘l videosi yig‘iladi. Model avtomatik annotation qiladi, odamlar esa tasdiqlaydi yoki tuzatadi. Bu semi-automatic yondashuv hisoblanadi.

Semi-automatic — yarim avtomatik. Model yordam beradi, odam tekshiradi.

Google kabi katta tizimlarda crowd-sourcing va model yordami birga ishlatilishi mumkin.

Crowd-sourcing — ko‘p odamlar yordamida data belgilash.

Startup loyihalarda Roboflow auto-label va manual correction ko‘p ishlatiladi. Startup — yangi kichik kompaniya.

Tibbiyotda ko‘p hollarda 100% shifokor annotation kerak bo‘ladi. Chunki xato label xavfli bo‘lishi mumkin. Tibbiy data’da sifat va ishonchlilik juda muhim.

Kurs yoki o‘rganish projectida eng qulay yo‘l:

200 ta rasm manual

↓

Roboflow auto-label

↓

Odam tekshiradi

↓

Model retrain qilinadi

Bu yondashuv vaqtni tejaydi va annotation sifatini saqlashga yordam beradi.

Amaliyot: LabelImg bilan Annotation va data.yaml

LabelImg bilan YOLO dataset tayyorlash uchun bir nechta asosiy bosqich bajariladi.

1. LabelImg oʻrnatish va ochish

Oʻrnatish:

```
pip install labelImg
```

Ochish:

```
labelImg
```

Bu buyruqlar bajarilgandan keyin LabelImg dasturi ochiladi.

2. Rasmlar papkasini tanlash

Open Dir orqali rasm papkasi tanlanadi. Masalan, kamida 50+ rasm boʻlgan papka tanlanadi.

Open Dir — rasmlar joylashgan papkani ochish.

Rasmlar quyidagicha boʻlishi mumkin:

```
images/train/dog001.jpg
```

```
images/train/dog002.jpg
```

```
images/train/cat001.jpg
```

3. Label saqlanadigan papkani tanlash

Change Save Dir orqali label fayllar saqlanadigan papka tanlanadi.

Masalan:

`labels/train/`

Bu juda muhim. Chunki rasm `images/train` ichida bo'lsa, label `labels/train` ichida bo'lishi kerak.

4. YOLO formatni tanlash

View → YOLO Format tanlanadi.

Bu juda muhim, chunki YOLO training uchun `.txt` label fayllar kerak bo'ladi. Agar noto'g'ri format tanlansa, label COCO yoki VOC ko'rinishida saqlanib qolishi mumkin.

YOLO format:

`class_id x_center y_center width height`

5. Box chizish va class kiritish

W tugmasi bosiladi. Keyin obyekt atrofida box chiziladi. Class nomi yoziladi va `Ctrl+S` bilan saqlanadi.

Jarayon:

W tugmasi

↓

obyekt atrofida box

↓

class nomi

↓

Ctrl+S

Agar rasmda bir nechta obyekt bo'lsa, har biri uchun alohida box chiziladi.

6. data.yaml faylini yozish

YOLO training uchun data.yaml fayli qo'lda yoziladi.

Namuna:

```
path: ./my_dataset
train: images/train
val:   images/val
nc: 2
names: ['dog', 'cat']
```

Bu yerda:

```
path → dataset asosiy papkasi
train → train rasmlar yo'li
val → validation rasmlar yo'li
nc → classlar soni
names → class nomlari
```

Agar names tartibi ['dog' , 'cat'] bo'lsa:

dog → 0

cat → 1

Label faylda 0 yozilgan bo'lsa, bu dog degani. 1 yozilgan bo'lsa, cat degani.

Har bir rasm uchun txt fayl

Har bir rasm uchun .txt fayl avtomatik yaratiladi.

Misol:

dog001.jpg → dog001.txt

Agar dog001.jpg ichida bitta dog bo'lsa, dog001.txt ichida shunday qator bo'lishi mumkin:

0 0.50 0.45 0.30 0.60

Agar .txt fayl bo'sh bo'lsa, bu rasmda obyekt yo'q degan ma'noni anglatadi. Object Detection datasetlarda negative image, ya'ni obyekt yo'q rasm ham bo'lishi mumkin.

Negative image — kerakli obyekt yo'q rasm. Bu modelga har doim obyekt bor deb o'ylamaslikni o'rgatadi.

Summary jadvali asosida asosiy tushunchalar

Pretrained Model

Pretrained Model — tayyor o'qitilgan model. Noldan boshlash o'rniga oldindan o'rgatilgan modeldan foydalaniladi.

Misol:

`yolov8n.pt`

Bu kabi model katta datasetlarda oldin o'qitilgan bo'ladi va keyin custom datasetga moslashtiriladi.

Fine-tuning

Fine-tuning — pretrained modelni o'z datasetga moslash.

Misol:

`200 rasm + 50 epoch = yaxshi boshlang'ich model`

Bu yerda model oldin umumiy featurelarni biladi, yangi dataset esa modelga maxsus vazifani o'rgatadi.

Transfer Learning

Transfer Learning — oldin o'rganilgan bilimni yangi vazifaga ko'chirish.

Computer Visionda transfer learning yaxshi ishlaydi, chunki quyi qatlam featurelari universal:

chiziq
shakl
burchak
tekstura

Bu featurelar ko‘p rasm turida uchraydi.

Backbone

Backbone — feature extraction qismi. Model rasm ichidan muhim belgilarni ajratadi.

Misollar:

CSPDarknet
C2f modules
ResNet
EfficientNet

Feature extraction — rasm ichidan foydali featurelarni ajratish.

Neck

Neck — multi-scale featurelarni birlashtiradigan qism.

Misollar:

FPN

PAN

PANet

Neck 52×52 , 26×26 , 13×13 kabi turli feature maplarni birlashtirib, kichik va katta obyektlarni topishga yordam beradi.

Head

Head — bounding box va class prediction chiqaradigan qism.

YOLO Head outputi:

$$S \times S \times (4 + 1 + C)$$

Bu yerda 4 — bbox koordinatalari, 1 — objectness score, C — classlar soni.

YOLO Label

YOLO Label formati:

class x_c y_c w h

Qiymatlar normalized 0–1 oralig'ida bo'ladi.

Misol:

0 0.5 0.45 0.3 0.6

Bu 0-class obyekt box markazi rasmning 50% chapdan, 45% yuqoridan joylashganini, box kengligi 30% va balandligi 60% ekanini bildiradi.

data.yaml

`data.yaml` — dataset konfiguratsiya fayli.

Ichida quyidagilar bo‘ladi:

```
path
train
val
nc
names
```

Bu fayl YOLOga dataset qayerda, nechta class bor va classlar nomi qanday ekanini aytadi.

Ultralytics

Ultralytics — YOLOv8 framework. U model train qilish, prediction olish, export qilish va turli CV tasklar bilan ishlashni osonlashtiradi.

Asosiy flow:

```
install
↓
model load
↓
train
↓
```

predict

↓

export

Labellmg

Labellmg — desktop annotation tool. Bepul, open-source va YOLO formatni qoʻllaydi.

Kichik dataset va beginner projectlar uchun juda qulay.

Roboflow

Roboflow — online annotation, auto-label va augmentation platformasi.

Katta loyiha, koʻp formatga export va auto-label kerak boʻlsa foydali.

Yakuniy umumiy tushuncha

YOLO bilan Object Detection project qilish uchun pretrained model, fine-tuning, YOLO architecture va dataset formati birgalikda tushunilishi kerak.

Toʻliq jarayon quyidagicha:

Pretrained YOLO model olinadi

↓

Custom dataset tayyorlanadi

↓

Rasmlarga bounding box annotation qilinadi
↓
YOLO label format yaratiladi
↓
data.yaml yoziladi
↓
Model fine-tuning qilinadi
↓
Model prediction qiladi
↓
Natija tekshiriladi
↓
Kerak bo'lsa annotation tuzatiladi va retrain qilinadi

Pretrained model noldan training qilishga qaraganda ancha qulay, chunki model umumiy visual featurelarni oldindan biladi. Fine-tuning orqali u yangi custom datasetga moslashadi.

YOLO architecture Backbone, Neck va Head qismlaridan iborat. Backbone rasm ichidan featurelarni ajratadi. Neck turli scale'dagi featurelarni birlashtiradi. Head esa bounding box, objectness score va class probabilitylarni chiqaradi.

YOLO dataset formati juda aniq tartib talab qiladi. `images/train` uchun mos `labels/train` bo'lishi kerak. Har bir rasmga mos `.txt` fayl bo'lishi kerak. Label fayl ichidagi qiymatlar `class_id` `x_center` `y_center` `width` `height` ko'rinishida normalized 0–1 oraliqda yoziladi.

Annotation uchun LabelImg, Roboflow va CVAT ishlatiladi. LabelImg oddiy va offline. Roboflow online, auto-label va

augmentation bilan qulay. CVAT professional va video annotation uchun kuchli.

Real loyihalarda manual va automatic annotation birga ishlatiladi. Avval kichik dataset qo'lda belgilanadi, keyin kichik model train qilinadi, model qolgan rasmlarni auto annotation qiladi, odam xatolarni tuzatadi va model full dataset bilan qayta train qilinadi. Bu Human-in-the-Loop yondashuvi hisoblanadi.

Lesson 8ning asosiy mazmuni shuki, YOLO bilan yaxshi Object Detection modeli qilish faqat modelni ishga tushirish emas. To'g'ri pretrained model tanlash, fine-tuning strategiyasini tushunish, backbone-neck-head arxitekturasini anglash, datasetni YOLO formatda tayyorlash, annotation sifatini nazorat qilish va data.yaml faylini to'g'ri yozish kerak bo'ladi.